

Parallel execution is how a game chain stays fast when thousands of players mutate inventories at once. The constraint is always the same: two transactions that touch the same mutable state cannot both commit as if they ran alone. Everything else is a strategy for finding that conflict cheaply and still using every core.

What “parallel” actually means

Consensus first produces an **ordered block** $T_1 \prec T_2 \prec \dots \prec T_n$. Execution must produce the **same state** as running that order serially. Parallelism is legal only when it is observationally equivalent to that serial order.

A conflict exists when two txs have overlapping write sets, or one writes a key the other reads (read–write). Game traffic is usually **low-contention at the object level** (many independent swords) and **high-contention at a few hubs** (one marketplace pool, one raid boss, one world clock). A good engine exploits the first and contains the second.

Four families of strategies

1. Pessimistic / declared access (Sealevel)

Idea: every transaction lists the accounts/objects it will read and write *before* execution. The scheduler builds a conflict graph, locks writable keys, and runs disjoint batches together.

text
tx.accounts = [sword_A (write), player_A (write), fee_payer (write)]

If tx1 and tx2 share no writable keys, they run on different cores. If they share a key, the later one waits or is deferred to the next wave.

Strength	Weakness
Almost no wasted work	Dev / client must list every account
Predictable scheduling	Missing an account → runtime reject
Strong under moderate contention	Shared hot accounts serialize the whole queue
Matches Rust “explicit ownership”	Extra tx size and client complexity

This is the Solana model. For games it is excellent when items are distinct accounts/objects

This is the Solana model. For games it is excellent when items are distinct accounts/objects. It is painful when a contract dynamically discovers what it will touch (loot tables, nested bags).

Rust shape: a lock table keyed by object ID, a work-stealing pool (rayon / custom), waves of ready txs.

2. Object-ownership / causal (Sui-style)

Idea: the *data model* is the scheduler. An object is either **owned** (one address) or **shared**.

- Owned-only txs: no global consensus, certify and execute immediately, fully parallel.
- Any shared object: go through consensus, then execute in causal order on that object's version chain.

Independence is structural, not inferred after the fact. Two players transferring two different NFTs never even enter the same queue.

Strength	Weakness
Fast path is almost free	Shared objects become the bottleneck
Matches game inventories	Must design objects so most actions stay owned
No speculative abort on the common path	Dynamic “touch whatever is in the bag” needs care
Consensus load drops with owned traffic	Versioned objects + wrapping/unwrapping complexity

This is the best *default* for a metaverse economy: a sword is an owned object; a marketplace AMM is shared.

Rust shape: object store with (id, version, owner) , two pipelines (owned certifier vs shared consensus), per-object queues.

3. Optimistic / Block-STM (Aptos, Sei, many EVMs)

Idea: assume independence. Execute all txs in the block in parallel against a **multi-version store**. Record each tx's read set and write set. Validate in consensus order: if a later tx read a key that an earlier tx wrote, abort and **re-execute** only the victim. Repeat until every tx validates. The committed result is still the serial order.

Developers write ordinary contracts. No access list.

text

execute speculatively → validate vs earlier writes → re-execute losers

Strength	Weakness
Best developer UX	Re-execution under contention wastes CPU
Works on account or object models	Needs a multi-version memory (RAM)
Composability stays “looks sequential”	Throughput collapses if everyone hits one pool
Adopted widely (Aptos, Polygon, Sei, Starknet)	Harder to reason about worst-case latency

Research follow-ons (RapidLane, NEMO) add **deferred objects** and **greedy commit for owned-only txs** so hot counters and owned assets abort less.

Rust shape: MVCC map Key → Vec<(version, value)>, collaborative scheduler, abort/re-execute loop. Fits Tokio + thread pool well.

4. Speculative / deferred execution (Monad-class)

Idea: split **ordering** from **execution**. Consensus agrees on the tx list first; execution runs asynchronously and in parallel (usually OCC on EVM bytecode). Nodes speculate on results for pipelining; state root lags the vote.

Useful if you want EVM compatibility. Less native for a greenfield Rust game chain unless you need Solidity studios.

How they behave on game workloads

Typical mix:

- 80–95% **owned-object** actions: transfer item, equip, consume potion, update private stats.
- 5–15% **shared**: listing on a market, joining a raid instance, mint from a shared recipe, world-event counter.
- Rare **hot keys**: one legendary listing, one boss HP bar, one faucet.

Workload	Sealevel (PCC)	Object-owned	Block-STM (OCC)
Independent loot transfers	Excellent	Excellent (bypass consensus)	Excellent (almost no aborts)

Nested inventory / dynamic reads	Awkward (must declare bag contents)	Natural if bag is an object	Natural
Single AMM / marketplace	Serializes on pool account	Serializes on shared object	Many re-executes
Boss HP decrements	One writable account → queue	Shared object queue	Abort storm unless deferred/commutative
Crafting that reads a shared recipe + owned inputs	Declare both	Recipe shared, inputs owned	Usually fine

Object-ownership wins the common path. OCC wins when contracts are messy and you refuse access lists. PCC wins when you can force a clean access list and want zero wasted cycles.

Hybrid strategy that fits the earlier architecture

Do not pick one. Compose them:

1. Classify at ingest

- Only owned objects → **fast path** (Sui-style, skip total order).
- Any shared object → **consensus path**.

2. On the consensus path, schedule with declared object IDs (Sealevel locks) because you already require object lists for the object model.

3. Inside a shared-object wave, run Block-STM for residual conflicts (two txs both touch market *and* a player inventory). Validation stays cheap because the write set is object-granular, not byte-granular EVM storage.

4. Make hot shared state commutative or deferred

- Counters, XP pools, “items remaining in a drop table”: use deferred / mergeable types (RapidLane idea) so increments do not abort each other.
- Boss HP: either shard instances (`BossInstance#{raid_id}`) or accept a single-object queue.

5. Keep gameplay off this engine. Parallel L1 execution is for *economy ticks*, not physics. The WASM coprocessor can run its own optimistic loop and post one attested result object.

That hybrid is close to NEMO: OCC + object model + greedy commit for owned-only txs.

Scheduler internals (what to implement in Rust)

A practical engine has four stages:

1. **Decode + fingerprint** — object IDs, owner vs shared, estimated gas.
2. **Partition** — union-find or coloring on the conflict graph; owned-only set vs shared clusters.
3. **Execute** — work-stealing pool; each worker has a private write buffer.
4. **Validate / merge** — in block order for OCC, or lock-order for PCC; produce a new object-version map.

Useful details:

- **Multi-version object store:** `ObjectId → [(version, owner, blob)]`. Readers pin a version; writers append.
- **Incarnation numbers** (Block-STM): each re-execute is a new incarnation so stale validations die.
- **Priority to unblockers:** run txs that release locks / produce objects other txs wait on first.
- **Determinism:** same block + same store → same final versions on every validator. No `HashMap` iteration order, no timers in contracts.
- **Gas metering per worker** so a fat craft recipe cannot starve a core.

Pipelining matters as much as intra-block parallelism: fetch signatures, load objects from disk, execute, and commit can overlap across blocks (Solana TPU / Monad deferred execution).

Contention is a product problem

No scheduler saves you if every player writes `GlobalMarketplace` every frame.

Design rules:

- Prefer **owned objects** and **per-player or per-instance shared objects** over one global singleton.
- Split markets by collection or shard order books.
- Use **events + indexers** for reads that do not need consensus (leaderboards).
- Charge **storage rent** so abandoned shared objects disappear.

- For must-be-global state, use **commutative updates** or batched settlement (“apply 10k damage ticks as one object write”).

Recommendation for the Rust game platform

Layer	Strategy
Default asset moves	Object-ownership fast path
Declared dependencies	Object ID lists on every tx (PCC scheduling)
Residual conflicts in a block	Block-STM-style OCC + MVCC
Hot counters / drops	Deferred / mergeable objects
Real-time combat	Off-chain WASM, commit results as objects
Implementation	Rust workers, explicit object store, no EVM unless a sidecar

Pessimistic declaration plus object ownership gives you a cheap, predictable common path. Optimistic re-execution is the safety net when two contracts surprise each other. That combination matches how a digital economy actually runs: millions of independent items, a handful of crowded rooms.